

Q2 - College ID Generator

Dean Yockey

My computer

My personal computer is a Lenovo ThinkCentre I bought for \$75 on eBay. It's running the Linux Mint operating system. When running `lscpu`, the first few lines read:

```
Architecture:          x86_64
CPU op-mode(s):        32-bit, 64-bit
Address sizes:         39 bits physical, 48 bits virtual
Byte Order:            Little Endian
CPU(s):                4
On-line CPU(s) list:   0-3
```

My computer has 4 CPUs.

q2.c

Because the problem of checking if a student ID is valid is very similar to the problem of checking if a circuit is satisfiable, my code for this question is very similar to the code in `SATI.c`. I admit I did copy much of the code.

The main task is performed in the `checkID` function. This function calls two other functions, `digitsum` and `consecutivedigits`, which check the sum of the digits and if two consecutive digits are the same, respectively.

`checkID` uses `sprintf` to convert the int `id` into an array of characters. This is because iterating through an array is a simpler way to iterate over digits than using powers of ten and division and all that.

The `consecutivedigits` function takes a variable amount of time, since a number like 110000 will abort very early, while a number like 123456 will iterate through the whole array. Similarly, `checkID` will abort early if `consecutivedigits` fails.

`digitsum` always iterates through the entire 6-digit array, so it takes constant time. However, `checkID` will abort earlier if the sum is 7, before it checks 11 and 13, due to short-circuit evaluation.

Thus, `checkID` execution time is quite varying, just like `SATI.c`'s task execution time, so the method is the same, with simple cyclic mapping.

Program result

The program found that 527787 IDs met the criteria.

Benchmarking

I created a bash file for benchmarking. It reads:

```
#!/bin/bash
echo "Q2" > "q2log.txt"
for p in {1..30}
do
    echo "P: $p" >> q2log.txt
    echo "P: $p" # so we can monitor progress
    for i in {1..20}
    do
        mpirun -n $p --oversubscribe q2 >> q2log.txt
    done
done
```

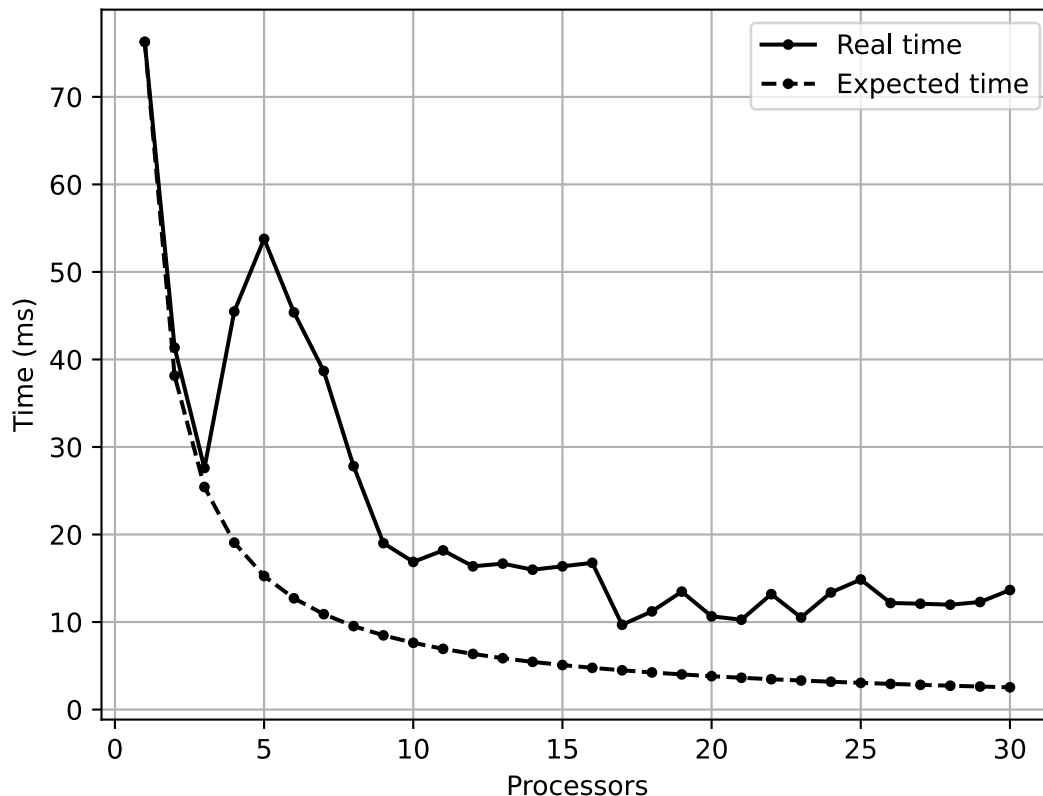
I ran the program 20 times for each process count. The output to `q2log.txt` looked like:

```
Q2
P: 1
T: 0.074175 s
T: 0.087600 s
T: 0.077718 s
T: 0.084953 s
T: 0.078401 s
T: 0.071897 s
T: 0.067269 s
T: 0.078525 s
T: 0.064668 s
T: 0.084758 s
T: 0.083995 s
T: 0.067355 s
T: 0.074425 s
T: 0.065048 s
T: 0.086018 s
T: 0.074042 s
T: 0.074104 s
T: 0.081815 s
```

```
T: 0.074902 s
T: 0.073949 s
P: 2
T: 0.037123 s
T: 0.040269 s
T: 0.043098 s
...
```

My results

To extract the results from the log file, and visualize them in a line graph, I wrote a python script. I used regex and matplotlib. This is my graph.



Takeaways

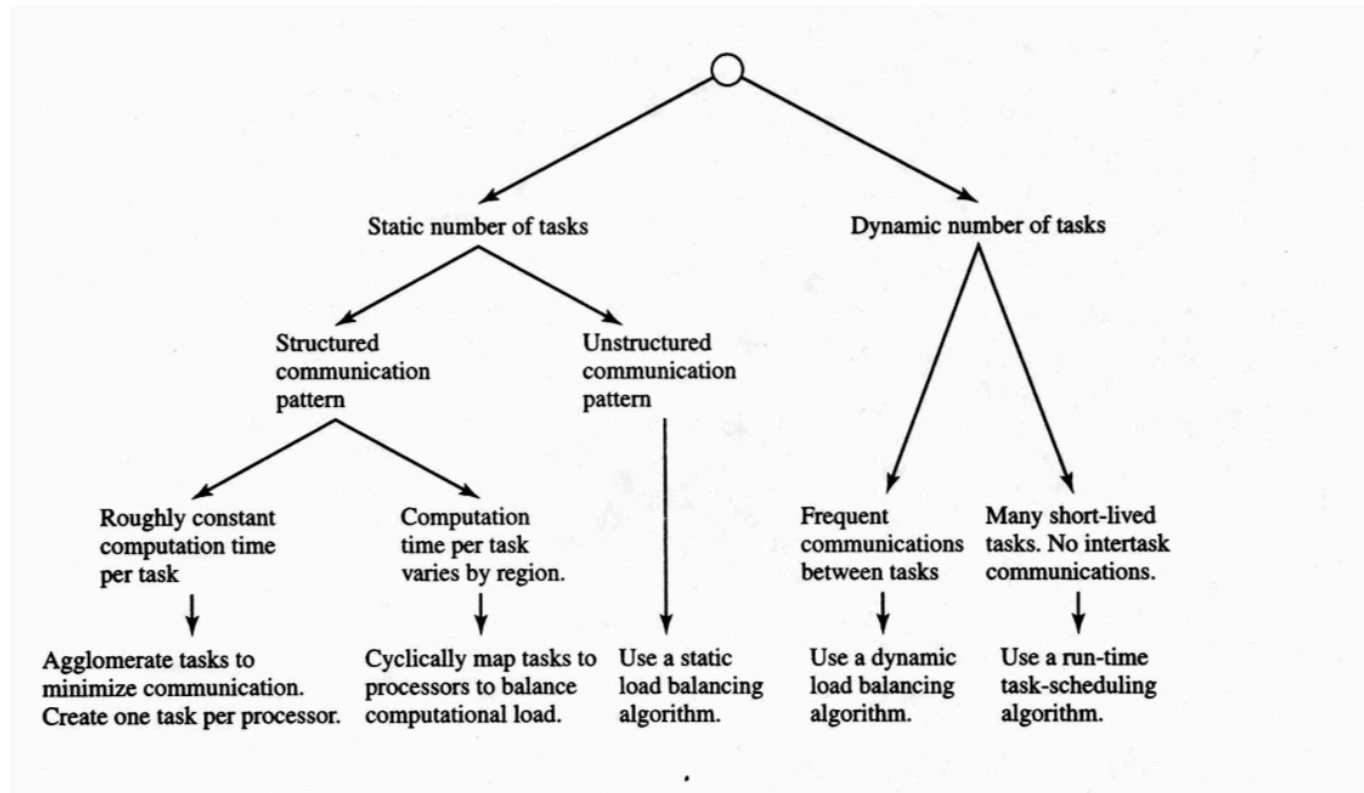
I expected the program to have peak performance at 4 processors, since that's the number of processors my computer actually has, but it actually has the best performance at 17.

There is a very large upward spike around 5 processes. 4, 5, 6, and 7 processes all do worse than 3 or less. Could this be the result of bad programming on my part? No, that's impossible!

My best explanation is that 4, 5, 6, 7, and even 8 processes simply result in a very lopsided distribution of work. They all still perform better than 1 processor. If the work distribution for 2

processors was 50%/50%, but the work distribution for 5 was 80%/5%/5%/5%/5%, it's clear how that would be slower.

Simply divvying up the numbers evenly is not the same as divvying up the work evenly. I expect there's a better method, but Foster's Flowchart suggests cyclic mapping for a task like this, which is what I did (as far as I can tell), so perhaps there is no real solution to this problem.



After the minimum value reached at 17, the real time gradually increases, as a trend. It's slight, but I would assume the time will slowly continue to increase as we go beyond 30.

The fact that the program is optimal at 17, far beyond my real processor count of 4, suggests that MPI's parallelization is very effective even on a single processor.